

PREVISÃO DE PREÇOS DO PETRÓLEO BRENT

Documentação Técnica de Projeto

Série Temporal · Prophet & LSTM · Aplicação Web Interativa

~15%

MAPE Prophet

1987

Início da série histórica

2

Modelos de ML

3

Páginas na aplicação

Autor: Francisco Costa Carneiro

Área: Ciência de Dados · Finanças · Machine Learning · Séries Temporais

Tecnologias: Python · Streamlit · TensorFlow/Keras · Prophet · Scikit-learn · Pandas · BeautifulSoup

Aplicação: <https://petrobrent.streamlit.app/>

Data: Maio de 2026

SUMÁRIO

1. Introdução e Contexto de Negócio
2. Objetivos do Projeto
3. Arquitetura Tecnológica
4. Fonte de Dados e Preparação
5. Suavização EWM e Pré-processamento para LSTM
6. Modelagem — Prophet
7. Modelagem — LSTM
8. Avaliação — Holdout Temporal e Métricas
9. Interface e Funcionalidades
10. Resultados e Conclusões
11. Anexo Técnico — Código-Fonte

1. Introdução e Contexto de Negócio

O petróleo Brent é a principal referência global para precificação do petróleo cru, impactando diretamente os mercados financeiros, as políticas energéticas de governos e as decisões estratégicas de empresas do setor. A volatilidade histórica do Brent — influenciada por fatores geopolíticos, variações de demanda e ciclos econômicos — torna a previsão de seus preços um problema de alta complexidade e relevância prática.

Este projeto implementa um sistema de previsão de séries temporais aplicado aos preços diários do petróleo Brent (USD/barril), utilizando dois modelos complementares de Machine Learning: o **Prophet** (modelo aditivo desenvolvido pela Meta) e o **LSTM** (Long Short-Term Memory — rede neural recorrente especializada em sequências temporais). Os dados são obtidos em tempo real via web scraping do portal IPEADATA e toda a pipeline é entregue em uma aplicação web interativa desenvolvida com Streamlit.

2. Objetivos do Projeto

Objetivo	Descrição
Primário	Construir um sistema de previsão de preços do petróleo Brent usando técnicas avançadas de séries temporais com avaliação honesta de desempenho.
Secundário	Implementar validação temporal correta (holdout temporal sem data leakage) e métricas calculadas sobre preços reais.
Técnico	Aplicar boas práticas de ciência de dados: suavização EWM, normalização MinMax, janela deslizante (look_back), holdout 80/20.
UX / Produto	Entregar interface web intuitiva que permita a qualquer usuário carregar dados, selecionar modelo, executar previsões e visualizar resultados.

3. Arquitetura Tecnológica

O projeto é estruturado em três camadas principais, seguindo boas práticas de arquitetura de sistemas de Machine Learning em produção:

Camada	Componente	Responsabilidade	Tecnologia
Apresentação	Interface Web	Formulário interativo, visualizações, feedback ao usuário	Streamlit 1.57
Dados	Web Scraping	Coleta em tempo real do IPEADATA via HTTP + parsing HTML	BeautifulSoup · Requests
ML / Analytics	Modelos de Previsão	Pré-processamento, treinamento (offline), predição online	TF/Keras 3.14 · Prophet 1.3
ML / Analytics	Normalização	Escalonamento MinMax das séries temporais	Scikit-learn 1.8
Persistência	Modelos Serializados	Modelos pré-treinados prontos para inferência	HDF5 (.h5) · JSON

Estrutura de Arquivos do Projeto

Arquivo / Pasta	Função
-----------------	--------

Home.py	Página inicial — descrição do projeto e instruções de uso
pages/2_Model.py	Carregamento de dados, seleção de modelo, execução e exibição de métricas
pages/3_Data_Visualization.py	Geração dos 4 gráficos analíticos com dados reais e preditos
models/lstm_model.h5	Rede LSTM pré-treinada (TensorFlow/Keras, formato HDF5)
models/prophet_model.json	Modelo Prophet serializado (Meta's Prophet, formato JSON)
Análise Exploratória - EDA.ipynb	Notebook de treinamento dos modelos (executado no Google Colab)
requirements.txt	Dependências do projeto com versões compatíveis com Python 3.13

4. Fonte de Dados e Preparação

Os dados históricos do petróleo Brent são obtidos em tempo real a partir do portal **IPEADATA** (Instituto de Pesquisa Econômica Aplicada), que disponibiliza a série EIA366_PBRENT366 — preço diário em USD por barril, desde 1987. A coleta é realizada via HTTP GET com parse da tabela HTML de ID `grd_DXMainTable`.

Etapas	Descrição
Fonte	IPEADATA — http://www.ipeadata.gov.br (série EIA366_PBRENT366)
Período	1987 até data atual (série histórica contínua, ~39 anos)
Frequência original	Dias úteis (sem fins de semana e feriados)
Parsing	BeautifulSoup — tabela HTML identificada pelo ID <code>grd_DXMainTable</code>
Formato de data	Brasileiro DD/MM/YYYY → convertido com <code>format='%d/%m/%Y'</code>
Conversão decimal	Vírgula decimal brasileira substituída por ponto (<code>replace(',', '.')</code>)
Resampling diário	<code>asfreq('D')</code> transforma para frequência diária contínua
Tratamento de ausentes	<code>bfill()</code> — backward fill para dias sem cotação (fins de semana)
Tipo de variável alvo	Preço do barril em USD (variável contínua, univariada)

5. Suavização EWM e Pré-processamento para LSTM

O preço diário do petróleo Brent apresenta alta volatilidade de curto prazo — variações bruscas causadas por eventos geopolíticos, anúncios de produção da OPEP e choques de demanda. Treinar um LSTM diretamente sobre essa série ruidosa prejudica a capacidade do modelo de capturar a tendência subjacente. A técnica de **Exponential Weighted Mean (EWM)** suaviza essa volatilidade sem eliminar a estrutura temporal.

EWM — Média Móvel Exponencial Ponderada

A fórmula EWM pondera observações recentes mais fortemente que observações antigas, com decaimento exponencial controlado pelo parâmetro α :

$$S_t = \alpha \cdot X_t + (1 - \alpha) \cdot S_{t-1}$$

Parâmetro	Valor	Efeito
-----------	-------	--------

α (alpha)	0.09	Suavização forte — peso alto para história; reage lentamente a choques
adjust	False	Modo recursivo puro (sem correção de viés inicial)
Aplicação	Apenas no LSTM	Prophet utiliza a série original (decomposição aditiva própria)
Métricas	Preços reais	Avaliação MAPE/RMSE/MAE sobre preços não suavizados (honesto)

Pipeline de Pré-processamento do LSTM

Etapa	Transformação
1. Suavização	EWM com $\alpha=0.09$ sobre a coluna 'y' (preço diário)
2. Reshape	Array 1D → matriz (-1, 1) para compatibilidade com MinMaxScaler
3. Normalização	MinMaxScaler: escala [0, 1] ajustado sobre toda a série
4. Divisão temporal	Split 80/20 — índice fixo, sem embaralhamento (sem data leakage)
5. Janela deslizante	look_back=5: cada amostra = 5 dias consecutivos → 1 previsão
6. Reshape LSTM	Formato (amostras, look_back, 1) — exigido pela camada LSTM

6. Modelagem — Prophet

O **Prophet** é um modelo de previsão de séries temporais desenvolvido pela Meta (Facebook AI Research), baseado em um framework aditivo que decompõe a série em três componentes: tendência não-linear, sazonalidades múltiplas e efeitos de feriados. Sua principal vantagem é a robustez a dados faltantes e a changepoints automáticos na tendência.

Aspecto	Detalhes
Algoritmo	Decomposição aditiva: $y(t) = g(t) + s(t) + h(t) + \epsilon(t)$
Sazonalidade	daily_seasonality=True — captura padrões intra-semana
Changepoints	Detectados automaticamente — adaptação a quebras estruturais
Treinamento	Série histórica completa (salvo e serializado em Colab)
Persistência	models/prophet_model.json (via prophet.serialize)
Previsão	make_future_dataframe(periods=N, freq='D') — N dias à frente
Validação	Holdout temporal: 20% finais da série (não vistos no treino)
MAPE (holdout)	~15.58% — calculado sobre preços reais
Incerteza	Intervalos de confiança disponíveis (yhat_lower, yhat_upper)

7. Modelagem — LSTM

As redes **LSTM (Long Short-Term Memory)** são redes neurais recorrentes com mecanismos de porta (forget, input, output gates) que permitem aprender dependências de longo prazo em sequências temporais — exatamente o padrão presente em séries de preços de commodities. O modelo foi treinado no Google Colab e salvo no formato HDF5 para inferência em produção.

Hiperparâmetro / Aspecto	Valor / Detalhe
Arquitetura	LSTM → Dense(1) — predição de um step à frente
look_back (janela)	5 dias — entrada: 5 preços suavizados → saída: 1 previsão
Épocas de treinamento	100 épocas (Google Colab)
Otimizador	Adam — taxa adaptativa, padrão para LSTM
Loss function	MSE (Mean Squared Error)
Entrada pré-treinamento	Série EWM suavizada com $\alpha=0.09$, normalizada MinMax [0,1]
Split de treinamento	80% treino / 20% holdout temporal (sem embaralhamento)
Persistência	models/lstm_model.h5 (carregado com tensorflow.keras.models.load_model)
Inferência (previsão futura)	Janela deslizante iterativa: prevê N dias consecutivos
Observação crítica	Prevê sobre série suavizada; métricas calculadas vs preços reais

8. Avaliação — Holdout Temporal e Métricas

A avaliação de modelos de séries temporais exige cuidado especial para evitar **data leakage** — o vazamento de informação futura para o conjunto de treino. Abordagens como validação cruzada aleatória são incorretas para séries temporais, pois violam a causalidade temporal. Este projeto aplica **holdout temporal estrito**: os últimos 20% da série são reservados como conjunto de teste, nunca vistos pelo modelo.

Protocolo de Validação

Aspecto	Implementação
Método	Holdout temporal — 80% treino 20% teste (índice fixo, sem shuffle)
Prophet	Merge entre previsão e dados reais do período de teste (inner join por data)
LSTM	Sequências do conjunto de teste geradas com numpy slicing (sem TimeseriesGenerator)
Métricas base	Preços REAIS do Brent (não suavizados pelo EWM)
Data leakage	Ausente — split por índice temporal, sem acesso ao futuro

Métricas de Avaliação

Três métricas complementares são calculadas e exibidas na interface:

Métrica	Fórmula	Interpretação
MAPE	$\frac{\text{mean}(y - \hat{y} / y) \times 100\%}{}$	Erro percentual médio — permite comparar modelos independente da escala do preço

RMSE	$\sqrt{\text{mean}((y - \hat{y})^2)}$	Raiz do erro quadrático médio em USD — penaliza erros grandes (outliers de preço)
MAE	$\text{mean}(y - \hat{y})$	Erro absoluto médio em USD — interpretação direta: desvio médio em dólares
Prophet MAPE	~15.58%	Medido no holdout temporal (20% finais da série histórica)

9. Interface e Funcionalidades

A aplicação web é desenvolvida com **Streamlit** em estrutura multi-página (MPA), onde o estado da sessão (`st.session_state`) compartilha dados entre as páginas. O fluxo completo é executado sequencialmente: coleta de dados → seleção do modelo → execução → visualização.

Página	Arquivo	Funcionalidades
Home	Home.py	Página de boas-vindas com fluxograma visual do projeto e instruções de uso
Model	pages/2_Model.py	Carregar dados via scraping · Selecionar modelo (Prophet ou LSTM) · Configurar período de previsão (slider 1–365 dias) · Executar modelo · Exibir tabela de métricas (MAPE, RMSE, MAE) com descrições
Data Visualization	pages/3_Data Visualization.py	Gráfico de linha histórico interativo · Gráfico real vs predito (desde dez/2023) · Barplot variação percentual por década (últimos 10 anos) · Boxplot distribuição anual de preços (1987–atual)

Fluxo de Execução

IPEADATA (scraping) → DataFrame Pandas → Seleção do Modelo → Execução (Prophet | LSTM) → Métricas (MAPE · RMSE · MAE) → Visualizações

10. Resultados e Conclusões

Comparativo dos Modelos

Critério	Prophet	LSTM + EWM
Tipo de modelo	Aditivo decomposicional	Rede neural recorrente (deep learning)
Sazonalidade	Capturada explicitamente	Aprendida implicitamente
Tendência	Modelagem não-linear com changepoints	Padrões capturados pela janela deslizante
MAPE (holdout)	~15.58%	Depende do período de teste (avaliado em tempo real)
Treinamento	Offline (Colab) — carregado em produção	Offline (Colab) — carregado em produção
Transparência	Alta — componentes interpretáveis	Baixa — caixa-preta
Previsão futura	N dias (autoregressivo interno)	N dias (janela deslizante iterativa)
Melhor para	Tendências de longo prazo + sazonalidade	Captura de padrões não-lineares

Conclusões e Próximos Passos

- Ambos os modelos são funcionais e entregam previsões com avaliação honesta via holdout temporal.
- A suavização EWM é fundamental para o LSTM — reduz o ruído sem eliminar a tendência estrutural.
- A aplicação Streamlit demonstra como pipelines de ML podem ser empacotados em interfaces acessíveis.
- Próximo passo técnico: retreinar periodicamente os modelos com dados mais recentes (drift temporal).
- Melhoria futura: adicionar intervalo de confiança ao LSTM via Monte Carlo Dropout.
- Melhoria futura: implementar modelo híbrido Prophet + LSTM (complementaridade dos modelos).
- Melhoria futura: incluir variáveis exógenas (câmbio, produção OPEP, índices de risco geopolítico).

11. Anexo Técnico — Código-Fonte

Coleta de Dados via Web Scrapping

```
def load_data():
    url = 'http://www.ipeadata.gov.br/ExibeSerie.aspx?...'
    response = requests.get(url)
    soup = BeautifulSoup(response.text, 'html.parser')
    tabela = soup.find('table', {'id': 'grd_DXMainTable'})
    df_base['data'] = pd.to_datetime(df_base['data'], format='%d/%m/%Y')
    df_base = df_base.asfreq('D').bfill() # resampling diário
```

Suavização EWM + Janela Deslizante para LSTM

```
alpha = 0.09
df['Smoothed_Close'] = df['y'].ewm(alpha=alpha, adjust=False).mean()

# Janela deslizante (substitui TimeseriesGenerator — removido no Keras 3)
def _make_sequences(data, look_back):
    X = np.array([data[i:i+look_back] for i in range(len(data)-look_back)])
    return X.reshape((X.shape[0], look_back, 1))
```

Métricas — Holdout Temporal sobre Preços Reais

```
actual_prices = df['y'].values[split + look_back: split + look_back + n]

mape = np.mean(np.abs((actual_prices - preds) / actual_prices)) * 100
rmse = np.sqrt(np.mean((actual_prices - preds) ** 2))
mae = np.mean(np.abs(actual_prices - preds))
```

→ O modelo LSTM prevê sobre a série suavizada (EWM), mas as métricas são calculadas comparando as previsões invertidas (escala original) com os PREÇOS REAIS do Brent — garantindo avaliação honesta sem inflação artificial do MAPE.